

COMPARATIVE ANALYSIS OF TCP TAHOE, RENO, AND CUBIC USING RELIABLE DATA TRANSFER OVER UDP

Submitted By:

Name: Patil Shradha Sushil

Roll No: 2403128

Course: Computer Networks

Academic Year: 2025-26

ABSTRACT

This project focuses on simulating congestion control in computer networks during reliable data transmission. A reliable data transfer system was implemented using UDP sockets in C language along with TCP-inspired techniques such as TCP Tahoe, TCP Reno, and TCP Cubic. Packet loss was artificially generated using transmission probability to simulate congestion and study protocol behavior under different network conditions. Performance metrics such as retransmissions, delay, and efficiency were recorded and analyzed using graphs. The project demonstrates how congestion control algorithms help maintain reliable and efficient communication in congested networks.

1. INTRODUCTION

Network congestion occurs when the amount of transmitted data exceeds available network capacity, resulting in packet loss, delay, and reduced performance. Modern transport protocols such as TCP use congestion control algorithms to dynamically regulate transmission rate according to network conditions.

This project demonstrates these concepts using a custom implementation over UDP sockets in C language. Since UDP does not provide reliability or congestion handling by default, features such as acknowledgments, retransmissions, sliding window protocols, and congestion window management were implemented manually. TCP Tahoe, Reno, and Cubic algorithms were compared under different network conditions.

2. OBJECTIVES

- Implement reliable data transfer using Go-Back-N
- Implement TCP Tahoe, Reno, and Cubic
- Simulate congestion using transmission probability
- Analyze retransmissions, delay, and efficiency
- Compare congestion control algorithms

3. BACKGROUND THEORY

TCP Tahoe resets the congestion window after congestion detection. TCP Reno improves recovery using Fast Retransmit and Fast Recovery. TCP Cubic provides faster congestion window growth for improved network utilization.

Reliable data transfer is implemented using a Go-Back-N sliding window protocol where multiple packets are sent before acknowledgments are received.

4. SYSTEM DESIGN

The project consists of sender and receiver modules communicating using UDP sockets. The sender manages congestion control and retransmissions while the receiver simulates packet loss and generates acknowledgments.

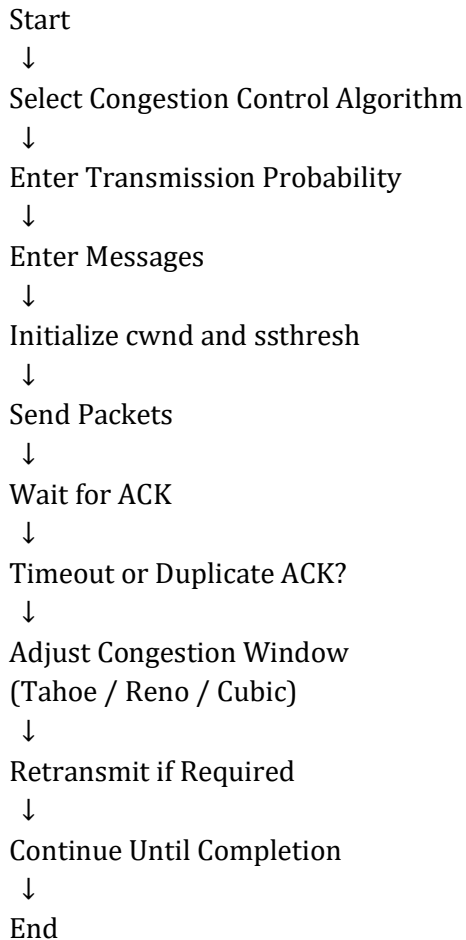
5. METHODOLOGY

Packets are transmitted using a sliding window protocol. Congestion is simulated using packet transmission probability. Depending on acknowledgments and timeout events, the sender dynamically adjusts the congestion window according to the selected TCP algorithm.

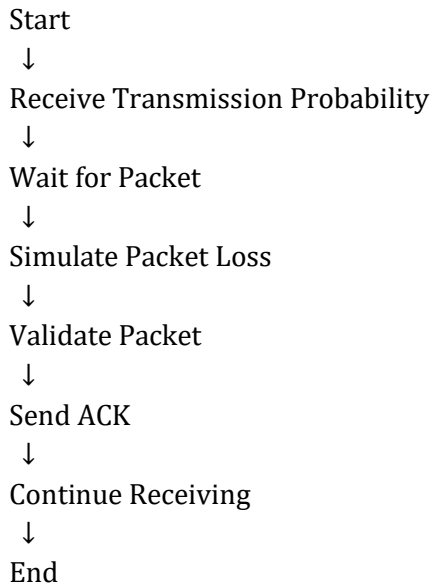
Experimental data such as retransmissions, delay, and efficiency are stored in CSV format and visualized using graphs.

6. FLOWCHARTS

6.1 Sender Side Flowchart



6.2 Receiver Side Flowchart



7. IMPLEMENTATION DETAILS

Programming Language: C

Communication Protocol: UDP

Operating System: Ubuntu/Linux

Features Implemented:

- Go-Back-N Sliding Window Protocol
- TCP Tahoe
- TCP Reno
- TCP Cubic
- Packet Loss Simulation
- Timeout Handling
- CSV Logging
- Graph Analysis

8. EXPERIMENTAL RESULTS AND ANALYSIS

The protocol was tested under different transmission probabilities for TCP Tahoe, Reno, and Cubic algorithms. Experimental observations showed that retransmissions and delay increase as congestion increases. TCP Cubic provided better efficiency and faster congestion window growth compared to Tahoe and Reno.

Figure 1: Retransmissions vs Transmission Probability

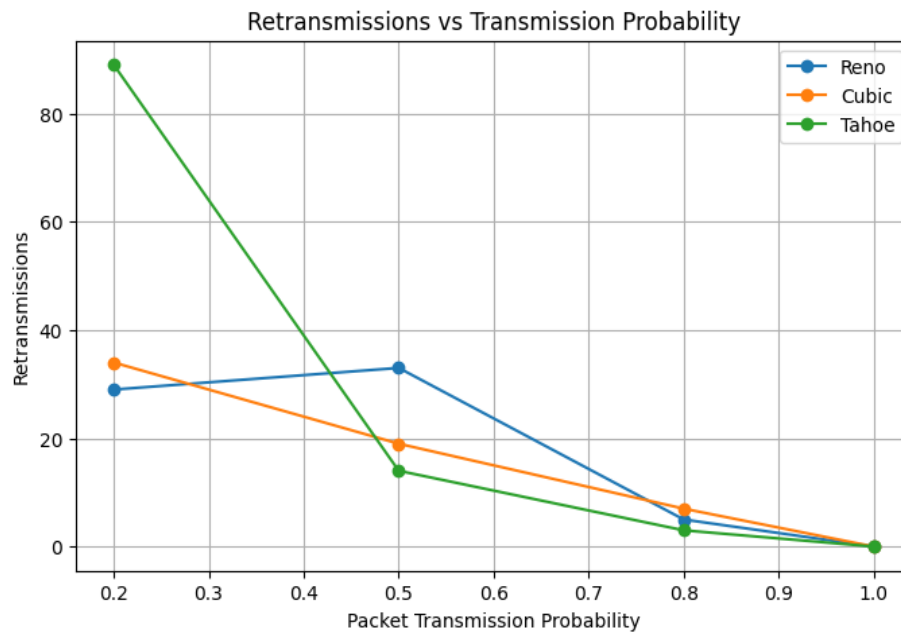


Figure 2: Delay vs Transmission Probability

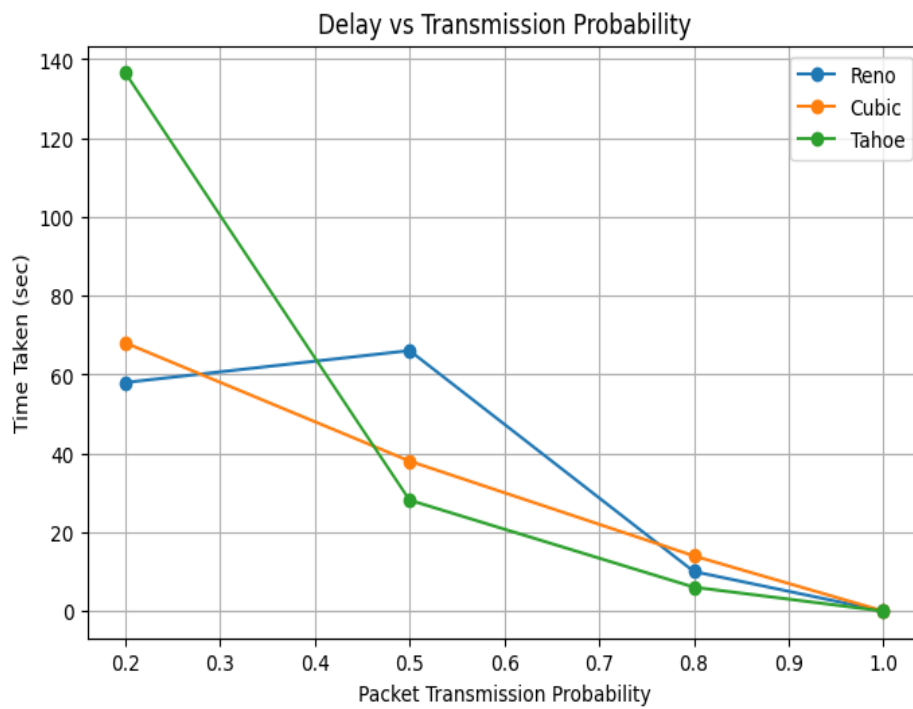
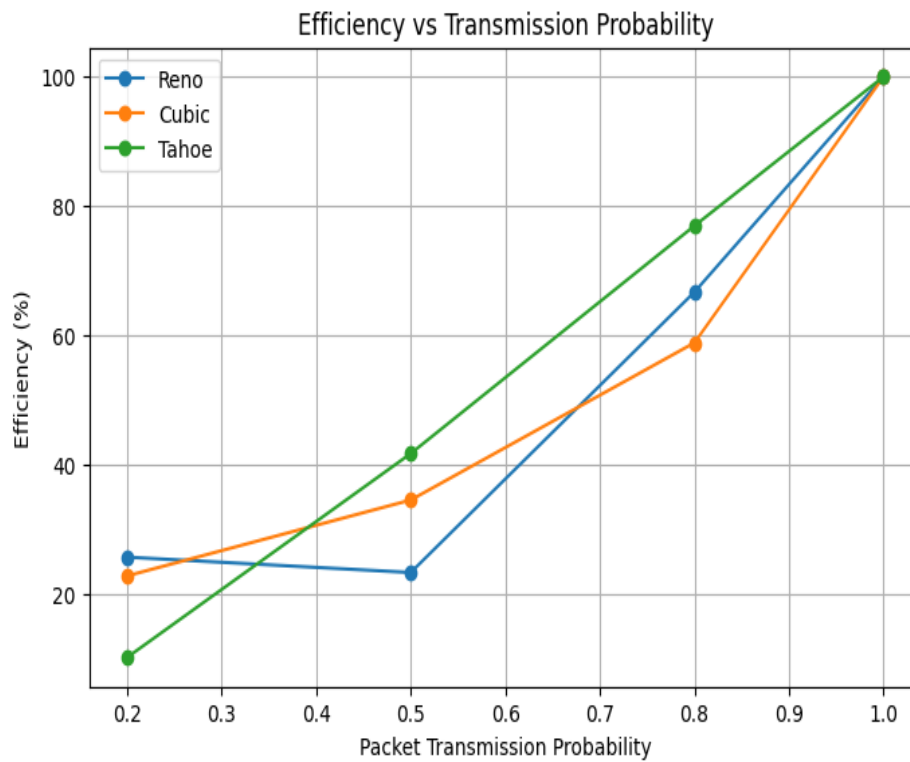


Figure 3: Efficiency vs Transmission Probability



9. CONCLUSION

This project successfully demonstrates congestion control using reliable data transfer over UDP sockets. The comparison of TCP Tahoe, Reno, and Cubic algorithms provides practical understanding of modern congestion control mechanisms and their behavior under varying network conditions.